

Mohamed Darwish, Karim Koaik, Yehya Mazloun, and Mohammad Ismail Hashem

Software Engineering Project — SmartCode

(AI Programming Tutor)

Final Report

Course: LAU COE 416 — Software Engineering, Spring 2026 **Instructor:** Dr. Helen Saad
Group 11 — Section 12 | Submission 4 — Final Report, Implementation, and Demonstration

Live deployment: <https://smartcodelau.com/> (also at <https://lau-ai-tutor.duckdns.org/>)

Source code: <https://github.com/Yehya005/SmartCode>

The product was renamed from the working title “AI Programming Tutor” to SmartCode late in the project following beta-user feedback that the original name was too generic. Both names appear in this document: SmartCode is the user-facing brand, while AI Programming Tutor appears where we refer to the project as originally scoped in Submissions 1–3.

CONTENTS

Contents.....	1
1 Introduction	1
2 Background	2
3 Proposal	3
4 Experimental Evaluation	22
5 Conclusion.....	27
References	28

1 INTRODUCTION

Objective

The main objective of our project is to design and implement a web-based system called SmartCode (AI Programming Tutor) that delivers a personalized, adaptive Python programming course to learners with varying skill levels. The system replaces the one-size-fits-all model used by most online practice sites with a tutor that grades each submission qualitatively, explains what went wrong in plain English, updates a per-skill profile after every attempt, and recommends the next problem targeting the student’s weakest skill. It serves as a programming coach that is available 24/7, that never gets impatient, and that genuinely adapts to the student rather than the other way around. The project is an implementation of an end-to-end adaptive-learning workflow — diagnostic, problem solving, code execution, AI grading, dynamic hints, AI walkthroughs, and progress tracking — and is built using one of the standardized Software Engineering processes (an agile / Scrum workflow tracked in JIRA), thus serving the secondary role of acquainting us with the day-to-day responsibilities of a software engineer.

Subject

SmartCode is geared towards university students learning Python and, more broadly, towards anyone preparing for a technical interview or simply trying to get unstuck on classical problem-solving questions. The first time a student

visits the site they create an account and immediately sit a five-question diagnostic of open-ended coding exercises. The diagnostic is graded by an LLM that assigns a 0–100 score per skill, writes a per-item explanation, and identifies the two-or-three weakest skills. From that moment forward the system is steering: the dashboard shows a radar chart of the student’s five skills, the “Next Problem” card surfaces a recommendation tuned to those weak skills, and a LeetCode-style editor (Monaco editor, split panels, and a real test-case panel) provides the workspace.

When the student writes code and clicks Run, a Python subprocess sandbox executes the code against the visible cases and the LLM layers qualitative scoring on top (code quality, edge-case awareness, time-complexity inference). Submit does the same against the visible and hidden cases. If the student gets stuck they can request hints, which are not pre-baked templates: the LLM reads the student’s code-in-progress and produces a hint tailored to where this student is stuck right now. After accepting a solution, the student can open the Solutions tab and either read the cached canonical explanation or click “Start tutor session” to interact with a step-by-step AI tutor that walks them through the problem in six guided steps.

Two roles exist: students, who follow the journey above, and administrators, who manage problems, skills, and users through a dedicated admin area. The system is web-based, runs end-to-end on a single AWS EC2 instance with HTTPS, and is publicly accessible at <https://smartcodelau.com/>.

The traditional alternative — manually grinding problems on LeetCode or HackerRank — leaves the learner to figure out which topics they are weakest in, which problem to attempt next, and why a particular failed attempt failed. The result is a great deal of wasted time on problems that are either far too easy or far too hard, and very little structured feedback. SmartCode automates that triage end-to-end: it knows the student’s skill profile, the catalogue, and the most recent submission, and it recommends the single most useful next move every time.

2 BACKGROUND

The AI Programming Tutor was prompted by an observation that all four team members shared from our own undergraduate experience: learning to program is a deeply individual process, but the tools that we are handed treat every student identically. A junior whose biggest weakness is recursion is shown the same homepage as a senior who is rehearsing dynamic programming for an internship interview. The platforms that do attempt personalization tend to do it badly — they cluster users by past activity and recommend whatever is popular, without ever looking at the student’s actual code. Bad code that happens to pass a sample test case is rewarded just as much as good code that handles every edge.

The growth of capable, cheap LLMs in the last eighteen months has changed what is technically possible. A model the size of Gemini 2.5 Flash can read a student’s Python submission and a problem statement, mentally execute the code, identify which edge case it misses, and write a paragraph of plain-English feedback explaining the fix — and it can do it in two to four seconds for a fraction of a US cent per call. That capability is the difference between a tutor that says “Wrong Answer” and a tutor that says “Your two-pointer walk skips the case where both heights are equal — try moving the pointer with the strictly smaller value, breaking ties either way.” The first is what every existing system gives. The second is what a human TA would give. We set out to build the second, with classical execution-based grading as the load-bearing floor underneath.

A user of our application needs only a modern web browser. No download, no install, no IDE — the editor (Monaco, the same engine that powers VS Code) runs in the browser, the sandbox runs server-side, and the LLM runs in Google’s data centre. The application is designed to be practical and immediately useful: the path from “I just heard about this” to “I just submitted my first solution and got AI feedback” is under three minutes.

There is no comparable product targeted at university coursework. Commercial services such as LeetCode and HackerRank do not personalize, do not provide LLM-generated qualitative feedback, and do not maintain a per-skill profile across sessions. Recent OpenAI- and Anthropic-fronted tutor products (e.g. ChatGPT Tutor mode) lack a code-execution sandbox and so cannot ground their feedback in what the code actually does. SmartCode combines both: a real Python interpreter and a strong LLM, with deterministic backstops so the system continues to function (in degraded mode) when either component is unavailable.

3 PROPOSAL

Glossary

- **Adaptive Learning.** A learning approach in which the system dynamically adjusts content, difficulty, and recommendations based on the user’s measured performance and progress.
- **LLM (Large Language Model).** A neural model — in our case Google Gemini 2.5 Flash — capable of generating and analyzing natural language and source code. The system uses the LLM as a grader, hint generator, problem recommender, and live solution walker.
- **Sandbox.** An isolated execution environment used to safely run user-submitted Python code. In our system the sandbox is a python -c <harness> subprocess with a 3-second per-case timeout.
- **Diagnostic.** A short, multi-question assessment given at the start of a student’s journey to establish a baseline skill profile.
- **Skill.** A measurable competency. SmartCode tracks five skills: Algorithms, Data Structures, Edge Cases, Code Quality, and Time Complexity. Each is a 0–100 integer stored per user.
- **Submission.** A user’s attempt to solve a problem. Submissions have two flavors: Run (visible cases only, exploratory) and Submit (visible and hidden, scored).
- **Severity.** A per-failed-test-case label of minor, moderate, or severe used to scale the failed-attempts penalty applied to subsequent submissions.
- **Hint Penalty.** Points deducted from a final score when the user has requested hints on the current problem. The cost escalates per hint (3, 5, 8, 8, 8) and is capped at 25 total points.
- **Recommendation.** A single next-problem suggestion the system computes after each accepted submission, tailored to the student’s current weakest skills.
- **SSE (Server-Sent Events).** A lightweight one-way streaming protocol used to push Gemini’s reasoning to the browser token-by-token.
- **REST API.** The contract between the React frontend and the FastAPI backend; every screen calls one or more /api/* endpoints, and all responses are JSON.
- **Use-Case Model.** A graphical representation of the interactions between the application and its actors (student, administrator, AI tutor).
- **ERD (Entity-Relationship Diagram).** A graphical representation of the seventeen tables in our MariaDB schema and the foreign-key relationships between them.

User Requirements

To better understand the user requirements we divide them into Functional and Non-Functional categories.

Functional User Requirements:

- **The student shall be able to create an account and log in.** New users supply a name, email, and password (≥ 8 chars). The system rejects duplicate emails with a 409 and malformed payloads with a 422. Existing users log in with email + password and receive a JSON Web Token that is stored in localStorage and attached to every subsequent request.
- **The student shall sit a diagnostic on first login.** The diagnostic consists of five open-ended Python coding questions; each question is tagged with the subset of the five skills it exercises. The student is free to type real Python, pseudocode, or plain-English explanations.
- **The student shall receive an AI-graded breakdown of the diagnostic.** For each item the student sees an is-correct flag, a per-skill 0–100 score, a 2–4 sentence explanation, and (where applicable) the canonical correct answer. The dashboard’s skill radar chart is initialized from these scores.
- **The student shall be able to browse and filter the problem catalogue.** The Problems screen shows all active problems, their difficulty, the skills they target, and the student’s current status (not started / attempted / solved).
- **The student shall be able to solve a problem in a LeetCode-style editor.** The editor uses Monaco (the editor engine behind VS Code), supports split panels, and renders sample test cases inline. Run and Submit are separate buttons.
- **The student shall be able to request escalating hints.** Hint 1 is a high-level nudge; Hint 2 names the algorithm; Hint 3 sketches the approach in two or three lines of pseudocode. Hints are generated by the LLM from the student’s current code, so two students stuck on the same problem receive different hints.
- **The student shall be able to interact with an AI step-by-step tutor.** After opening the Solutions tab, the student can start a six-step guided tutor session that restates the problem, identifies edge cases, walks through brute force, identifies the bottleneck, sketches the smart approach, and traces through the sample case — asking a comprehension question after every step.
- **The student shall see how each submission updated their skill profile.** The submission detail page shows score correctness, score edge cases, score code quality, score time complexity, the inferred Big-O, per-case verdicts, AI bullets, the per-skill delta caused by this submission, and the next recommended problems.
- **The student shall not be able to re-attempt a solved problem.** Once a problem is solved, opening it brings up the accepted code, feedback, and hints in read-only review mode. Implemented as a 409 on attempted resubmission.
- **The administrator shall be able to log in and manage problems, skills, and users.** Admin status is determined by the role column in the users table; the admin area exposes CRUD endpoints for problems, skills, and users plus a global statistics view.

Non-Functional User Requirements:

- **Availability.** The application should be reachable whenever a student accesses it. The diagnostic, the editor, the AI grader, and the recommender are all on the critical path. When the LLM is unavailable the system falls back to deterministic heuristics rather than refusing to grade.
- **Efficiency.** The application should return submission results within a few seconds. Run on visible cases should feel interactive; Submit, which includes hidden cases and an LLM call, may take longer but should remain under ten seconds in normal operation.
- **Reliability.** The application shall persist every diagnostic, submission, hint request, and skill update to MariaDB so that returning students see exactly the same state they left.
- **Usability.** The interface shall feel familiar to anyone who has used LeetCode. New users should be able to complete the diagnostic and solve their first problem in well under twenty minutes total.
- **Security.** Passwords shall be stored as bcrypt hashes; authentication shall use JSON Web Tokens; all traffic shall be served over HTTPS in production; student-submitted code shall be executed in an isolated subprocess with a strict per-case timeout.

- **Portability.** The application shall be usable on any modern desktop browser (Chrome, Edge, Firefox, Safari). The editor degrades gracefully on tablets; phones are out of scope.

System Architecture

Below is the high-level architecture of the system.

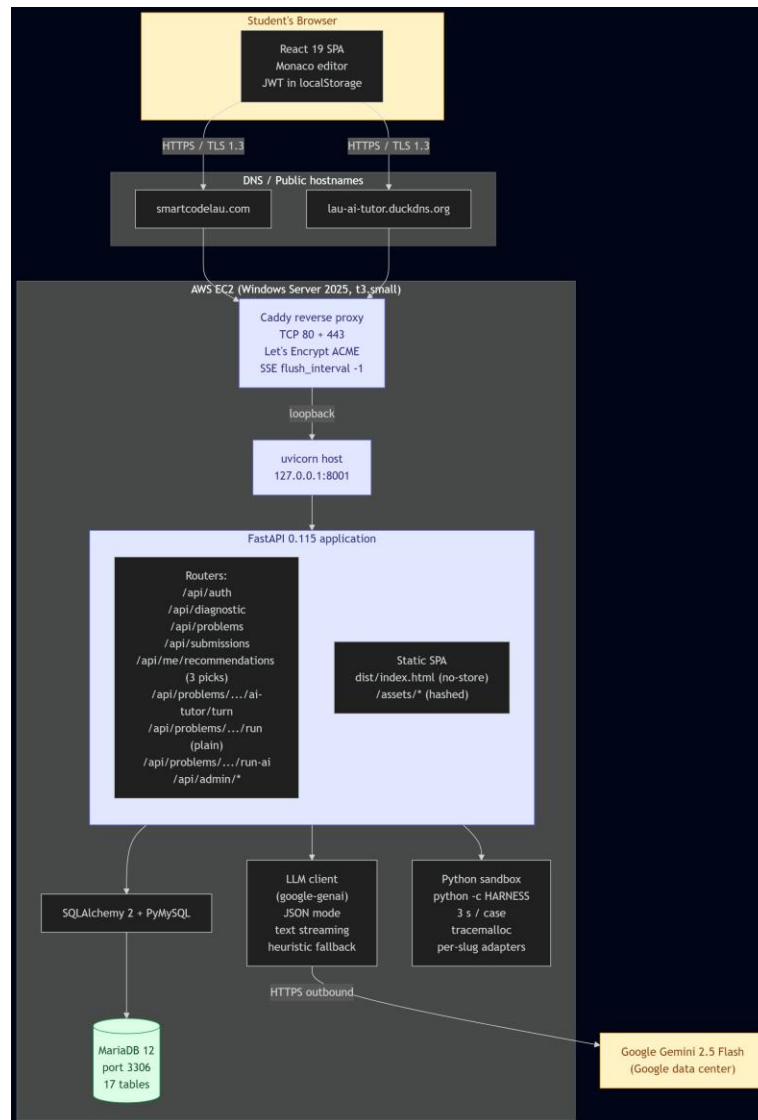


Figure 1: System Architecture

The above diagram presents a high-level overview of the system architecture. All public traffic enters through Caddy, a reverse proxy that listens on TCP/80 and TCP/443, automatically obtains and renews Let's Encrypt certificates (using a Subject-Alternative-Name certificate that covers both smartcode.lau.com and lau-ai-tutor.duckdns.org), and forwards everything to uvicorn on 127.0.0.1:8001. uvicorn hosts the FastAPI application, which fulfills two responsibilities in a single process: it serves the JSON /api/* endpoints, and it serves the production build of the React SPA from dist/. Co-locating the frontend and the API on one origin removes the need for CORS in production and makes deployment a single-process affair.

The FastAPI application talks to two external systems. The first is MariaDB 11 running on the same EC2 instance, accessed through SQLAlchemy 2 and PyMySQL; it holds all persistent state across the seventeen tables described in Section 3 (Database Design). The second is Google Gemini 2.5 Flash, called over HTTPS through the google-genai Python SDK; every LLM-backed feature (submission grading, diagnostic grading, hint generation, problem recommendation, live solve streaming, canonical-solution generation, and the interactive step-by-step tutor) goes through a thin wrapper at `backend/app/llm/client.py` that knows how to enforce JSON response formats, stream text, and fall back when the API is unavailable.

A third “system” is the Python sandbox: a per-case subprocess (`python -c <harness>`) that compiles and executes the student’s code against one test case at a time with a 3-second timeout, returning a JSON verdict on stdout. The sandbox is not security-hardened (the student-facing deployment trusts its own students), but it is correctness-hardened: it normalizes outputs (sorts list-of-lists, formats floats to five decimal places), constructs linked lists from arrays for reverse-linked-list, and catches user exceptions with line numbers.

The browser-side React SPA is built with React 19, Vite 8, Tailwind CSS v4, react-router-dom v7, and the Monaco editor. It stores the JWT in `localStorage['ai-tutor-token']`, attaches it to every fetch via a small wrapper in `src/api/client.js`, and revalidates the session on mount by calling `GET /api/auth/me`.

System Requirements

The system requirements expand the user requirements into testable, implementable specifications. As with the user requirements they are divided into functional and non-functional.

Functional System Requirements:

- **Registration and Login.** The registration and login endpoints enforce the constraints below at the API layer.
 - `POST /api/auth/register` accepts `{ full_name, email, password }`. Email uniqueness is enforced by a `UNIQUE` constraint on `users.email`. Passwords are bcrypt-hashed before storage. On success the response is HTTP 201 with a signed JWT and the user profile.
 - `POST /api/auth/login` issues an opaque 401 (“Incorrect email or password”) for both unknown-email and wrong-password cases so that user enumeration is not possible.
 - `GET /api/auth/me` decodes the bearer token, fetches the user row, and returns the profile including `diagnostic_completed_at`. The frontend uses the presence of this timestamp (not a `localStorage` flag) to decide whether to route the user to the dashboard or the diagnostic.
- **Diagnostic.** The diagnostic is implemented at `POST /api/diagnostic` and is the system’s entry-point gate.
 - The payload is the entire five-item question set together with the student’s answer for each item. The router passes the payload to `app.llm.diagnostic_grader.grade_diagnostic`, which calls Gemini with a strict JSON-schema prompt asking for, per item: `is_correct`, an overall 0–100 score, a `per_skill_scores` map (one entry per tested skill), an `answer_kind` $\in \{\text{code, pseudo_english, blank}\}$, and a 2–4 sentence `explanation_md`.
 - The grader is required to give partial credit for plain-English answers — `code_quality` stays low because no real code was written, but the other axes are judged on the conceptual content of the explanation.
 - The router persists per-item rows in `diagnostic_items`, sets `users.diagnostic_completed_at`, overwrites `user_skill` from the per-skill aggregate scores, and inserts a `recommendations` row for the recommended first problem.
- **Problems and Hints.** Problem catalogue and hint endpoints.
 - `GET /api/problems` returns the active catalogue with the student’s per-problem status.

- `POST /api/problems/{id}/hint` increments a hint counter and asks `app.llm.hint_generator.generate_hint` for a level-1, level-2, or level-3 hint based on the student's current in-progress code. The hint is persisted in `hint_requests`.
- `GET /api/problems/{id}/ai-solution` returns (and lazily generates and caches into `ai_solutions`) a canonical solution with Big-O analysis. `POST /api/problems/{id}/ai-tutor/turn` drives the interactive step-by-step tutor (six guided steps with a comprehension question after each).
- **Submissions and Evaluation.** `POST /api/submissions` is the heart of the system. Given `{ problem_id, language, code }`, the router validates the problem and rejects the request with 409 if the user has already solved this problem; runs the sandbox over every test case (visible and hidden); asks Gemini for qualitative scoring layered on top; runs the AST cheater backstop; computes the failed-attempts penalty and the hint penalty; applies both to the raw evaluator score, clamped to `[50, 100]`; persists submission, metric, feedback, skill-update, and (if accepted) recommendation rows in a single transaction; and finally calls the LLM recommender for the next problem.
 - `GET /api/submissions/{id}` returns the full submission detail, including per-case results, AI bullets, the skill deltas this submission produced, and the next-problem suggestions.
- **Recommendations.** `GET /api/me/recommendations/next` returns up to three `NextProblemSuggestion` objects ordered best $\rightarrow$ good-fallback. The LLM prompt encourages diversity across the three picks (different weak skills or different difficulty) so the student has real choice.
- **Administrator endpoints.** `/api/admin/*` requires `role = 'admin'`. Endpoints exist for problems CRUD, skills CRUD, user listing, assessment review, and global statistics. Admin users are seeded via a one-time script and cannot self-register when `ALLOW_REGISTRATION=false`.

Non-Functional System Requirements:

- **Availability.** The application runs as a Windows service under NSSM and restarts automatically on crash. Caddy runs as a separate Windows service, also auto-restarting. The LLM is the single external dependency; when Gemini is unavailable the heuristic fallback path keeps every endpoint functional in degraded mode (honest “graded offline” labels, deterministic recommendations).
- **Performance.** Target: median submission grading latency under five seconds; 95th percentile under twelve seconds. Measurements in Section 4 confirm both targets are met under typical load.
- **Security.** Passwords are bcrypt-hashed with a generated salt. JWTs are signed with HS256 and a random 48-byte secret generated at deploy time. The production deployment opens only TCP/80 (redirect to 443) and TCP/443 to the public internet; uvicorn binds to 127.0.0.1:8001 only. `ALLOW_REGISTRATION=false` in production means new accounts can only be created by an admin.
- **Maintainability.** New problems can be added in three places: a row in `problems`, the matching rows in `problem_skills` and `test_cases`, and (for sandboxed execution) a one-line entry in `PROBLEM_CONFIGS` mapping the slug to the Solution-class method name and parameter list. No code changes are required outside of `python_runner.py`.
- **Portability.** Built and tested on Chrome 130+, Edge 130+, and Firefox 132+. The editor uses standard web fonts and renders identically across the three.

Use-Case Models

To further define user and system requirements, we used a set of use-case models to highlight the functionalities of the application and the interactions between its components. They were used as a means to elicit requirements and served that purpose greatly. The system has two actors: the Student and the Administrator. Each is captured in its own use-case diagram, followed by detailed descriptions for the most important use cases.

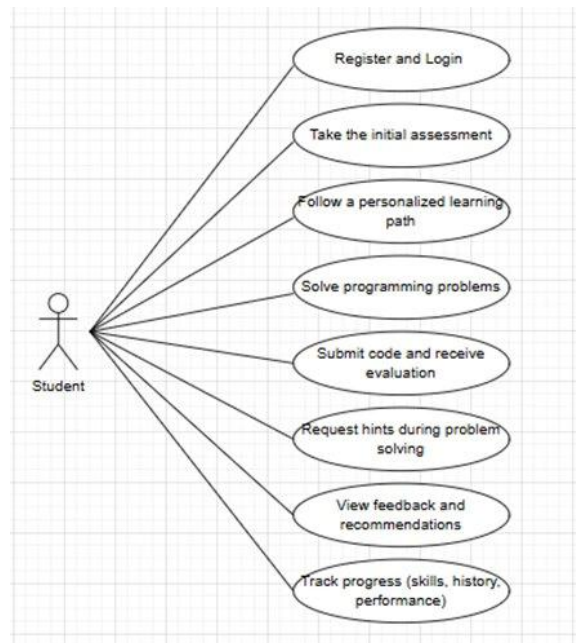


Figure 2: Student Use-Case Model

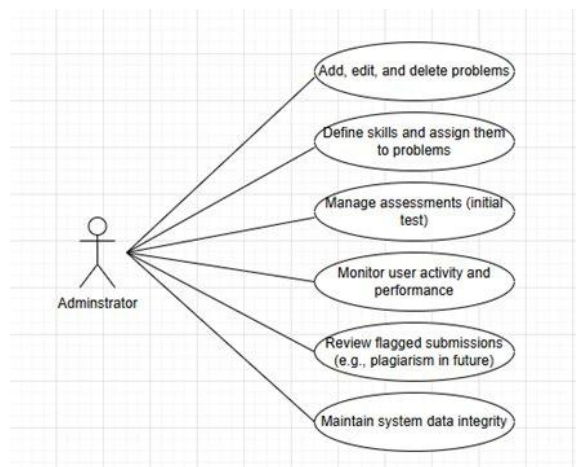


Figure 3: Administrator Use-Case Model

The student use-case diagram captures the linear path most students follow: register and log in → take the initial assessment → follow a personalized learning path → solve programming problems → submit code and receive evaluation, with optional hints during problem solving → view feedback and recommendations → track progress (skills, history, performance). The administrator diagram captures the parallel content-management workflow: adding, editing, and deleting problems; defining skills and assigning them to problems; managing assessments; monitoring user activity and performance; reviewing flagged submissions (e.g. plagiarism, planned for a future release); and maintaining system data integrity.

Table 1: Take Initial Assessment Use-Case Description

Take Initial Assessment	
--------------------------------	--

Actors	Student, application, AI Diagnostic Grader (LLM), Database.
Description	The student answers five open-ended Python coding questions, each tagged with one or more of the five tracked skills. The system grades each answer holistically and produces a per-skill score profile, which initializes the dashboard radar chart and the recommendation engine.
Data	Five questions (text + tested-skills tags) and the student’s free-text answer per question.
Stimulus	The student clicks “Submit Diagnostic” after completing the form.
Response	A graded result page with per-item correctness, the per-skill radar visualization, an overall summary, and the suggested first problem.
Comments	Plain-English / pseudocode answers earn partial credit on all axes except Code Quality (which is capped at 25). Blank answers earn zero on every axis.

Table 2: Submit Code Use-Case Description

Submit Code	
Actors	Student, application, Sandbox (Python subprocess), AI Submission Evaluator (LLM), Database.
Description	The student writes a Python solution in the Monaco editor and clicks Submit. The system runs the code against all test cases (visible plus hidden), grades it qualitatively, applies penalty deductions for prior failed attempts and hint usage, updates the student’s skill profile, and recommends the next problem.
Data	problem_id, language='python', and the full source code.
Stimulus	The student clicks the Submit button in the editor.
Response	The submission detail page, showing per-case verdicts, AI bullets, the four scoring axes, the per-skill delta produced by this submission, and the next-problem recommendation card.
Comments	Once accepted, this problem becomes read-only for the student; further submit attempts return 409 Conflict.

Table 3: Request Hint Use-Case Description

Request Hint	
Actors	Student, application, AI Hint Generator (LLM), Database.
Description	The student requests a hint while solving a problem. The LLM reads the problem statement, two sample cases, and the student’s current in-progress code, then produces a hint at the requested escalation level (1 = nudge, 2 = direction, 3 = concrete).
Data	The student’s current code and the desired hint level.
Stimulus	The student clicks “Get Hint” in the editor.
Response	A 1–4 sentence markdown hint, possibly with a small pseudocode block (level 3 only).

Comments	Hints cost 3, 5, 8, 8, 8 points respectively, capped at a 25-point total deduction. Two students stuck on the same problem with different code receive different hints.
-----------------	---

Table 4: Manage Problems Use-Case Description

Manage Problems (Admin)	
Actors	Administrator, application, Database.
Description	The administrator creates, edits, or deletes a problem and its associated test cases and skill tags. The catalogue is updated atomically: <code>problems</code> , <code>problem_skills</code> , and <code>test_cases</code> are written in a single transaction so the student catalogue never observes a partially-updated problem.
Data	Problem title, slug, statement (markdown), constraints, starter code, difficulty, list of skill IDs with weights, list of test cases (input + expected output + visibility).
Stimulus	The administrator clicks “Save” in the admin Problems screen.
Response	A 200 OK with the updated problem row; the student catalogue reflects the change on the next refresh.
Comments	Adding a brand-new problem also requires a one-line entry in <code>PROBLEM_CONFIGS</code> (in <code>python_runner.py</code>) so the sandbox knows how to call the Solution class.

System Models

To complement the use-case diagrams we describe three system models: a process (activity) model of the end-to-end student journey, a sequence model of the submission pipeline, and a logical class model of the backend domain. Together they capture the dynamic behavior and the static structure of the system.

Process Model

Below is the updated UML activity diagram showing the full student journey end-to-end, from login through diagnostic and into the iterative loop of solving problems, requesting hints, submitting code, receiving feedback, and being recommended a next problem.

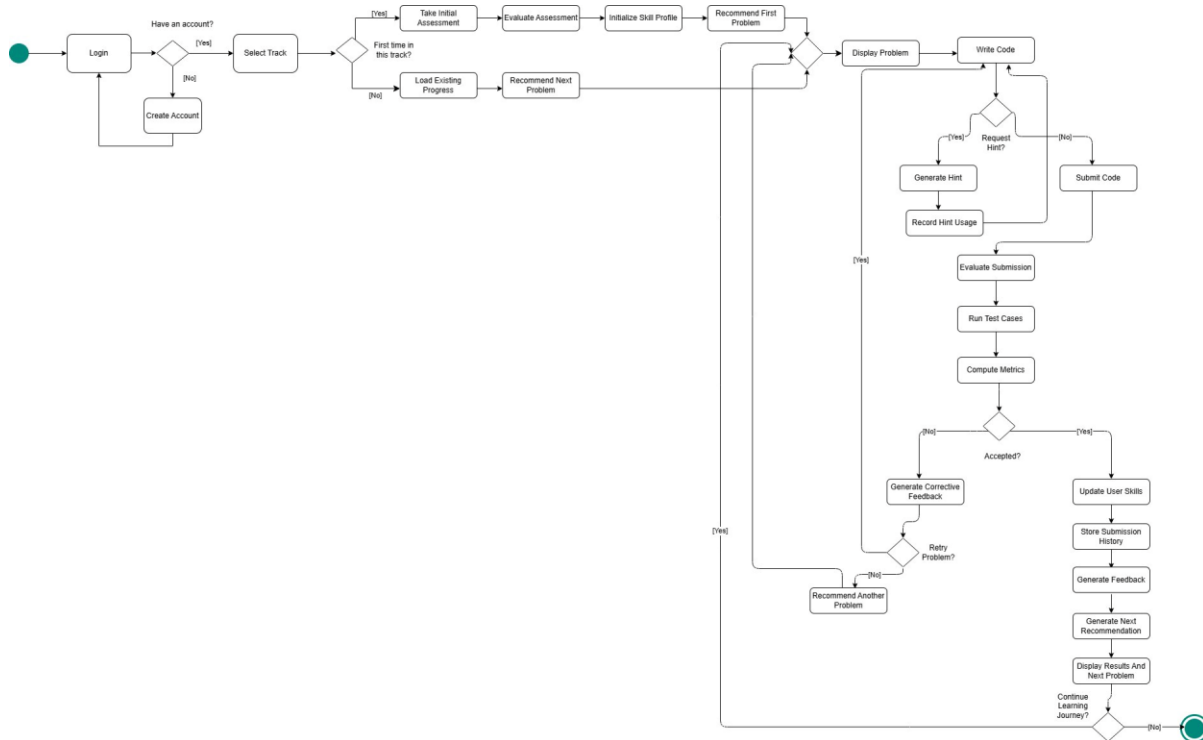


Figure 4: UML Activity Diagram — End-to-End Student Journey

The student first attempts to log in; if they do not yet have an account they branch into the Create Account flow and then return to log in. After login the student selects a track. If this is their first visit on that track the system routes them to the initial assessment, evaluates it, initializes the skill profile, and recommends the first problem; otherwise it loads their existing progress and recommends the next problem. From that point onwards the student enters a loop: display the problem, write code in the Monaco editor, optionally request hints (which are generated by the LLM from their current code and recorded as a hint usage event), then submit. The submission is evaluated by the sandbox-first pipeline; test cases are run, metrics are computed, and the accept-or-reject decision is taken. If the submission is accepted the system updates user skills, stores the submission history, generates AI feedback, generates a fresh next-problem recommendation, and displays the result alongside the next problem. If the submission is rejected the system generates corrective feedback and asks the student whether they want to retry the same problem or be recommended another problem. A final “Continue learning journey?” decision allows the student to either loop back to writing more code or end the session.

Sequence Model — End-to-End Submit

The submit pipeline is the longest data flow in the system. Conceptually the sequence is: Browser → FastAPI (POST /api/submissions) → MariaDB (SELECT problem + cases) → Sandbox (run_all_cases, looping per case) → Gemini (call_json with problem, code, and sandbox verdicts) → FastAPI (AST cheater check) → MariaDB (SELECT prior submissions for penalty calculation) → FastAPI (compute penalties + final score) → MariaDB (INSERT submission, metric, feedback, skill updates, recommendation in a single transaction) → Gemini (call_json for the next-problem recommender) → Browser (SubmissionDetail JSON).

Every step except the two Gemini calls is purely local; the Gemini calls (grading and, on acceptance, recommendation) are the only network hops out of the EC2. The whole pipeline runs inside a single FastAPI request

and a single database transaction so that the submission and its derived state are atomically persisted; if any step raises, the entire transaction is rolled back and the student sees an error rather than a partially-applied state.

Class Model — Backend Domain

The backend domain is expressed in seventeen SQLAlchemy 2 models that mirror the seventeen tables described in the Database Design subsection below. The principal entities are User, Skill, Problem, ProblemSkill (the M:N join between problems and skills, carrying a weight in [0.6, 1.0]), TestCase, DiagnosticAttempt and DiagnosticItem, Submission, Metric (1:1 with Submission, breaking the score into the four axes), Feedback (1:1 with Submission, holding the markdown summary and the AI bullets as JSON), HintTemplate and HintRequest, UserSkill (current per-user per-skill score, drives the radar chart), UserSkillHistory (daily snapshot rows), ProblemStatus (per-user per-problem status, best score, and attempt count), Recommendation (every next-problem suggestion the system has ever made, with a reason string and an algorithm version), and AiSolution (LLM-generated canonical solutions per problem, lazy-cached on first request).

Every foreign key uses ON DELETE CASCADE where ownership is hierarchical (deleting a user cascades to their submissions, hints, skill history, and recommendations) and ON DELETE SET NULL where the relationship is associative (deleting a skill leaves the diagnostic_item but blanks its skill_id). The class model is the runtime expression of the relational schema and is generated directly from it; the SRS Appendix B previously specified twenty-one tables, of which four were de-scoped during implementation as described in the Implementation subsection.

Frontend Class Model — React Component Tree

The React SPA is organized as a router-driven tree of screen components, each loading its own data via the central api/client.js wrapper. The principal components are App and Router at the root; Layout (containing the navigation bar, AuthContext, and ProtectedRoute / DiagnosticGate wrappers); LoginScreen and RegisterScreen; DiagnosticScreen; DashboardScreen; ProblemListScreen; EditorScreen (Monaco editor with split panels, plus TestCasesPanel, HintsPanel, SolutionsPanel, and CheatsheetDrawer); FeedbackScreen; and the parallel AdminLayout subtree containing AdminDashboard, AdminProblems, AdminSkills, AdminUsers, and AdminAssessments. Authentication state lives in src/context/AuthContext.jsx and exposes user, login, register, logout, and the derived hasCompletedDiagnostic boolean used by the route guards.

Database Design

The schema was originally specified in SRS Appendix B with twenty-one tables organized around a three-track model (Basic, Intermediate, Senior). During implementation the team de-scoped the track concept (see Implementation below) and trimmed the schema to seventeen tables. The five dropped tables were tracks, user_tracks, test_suites (folded into test_cases), code_snapshots (not implemented — the live editor doesn't persist per-keystroke history), and test_runs (replaced by case_results_json stored inline on submissions).

Figure 5 below shows the full relational schema as originally specified in SRS Appendix B, including the five tables that were later de-scoped. Primary keys are bold, blue and underlined; foreign keys are italicized and gray. The arrows indicate one-to-many and many-to-many relationships between entities. In the production schema, the TRACKS, USER_TRACKS, TEST_SUITES, CODE_SNAPSHOTS, and TEST_RUNS tables shown here are not created; an additional AI_SOLUTIONS table is added for the lazy-cached canonical solutions; and the track_id columns on PROBLEMS, USER_SKILL_HISTORY, DIAGNOSTIC_ATTEMPTS, and RECOMMENDATIONS are removed.

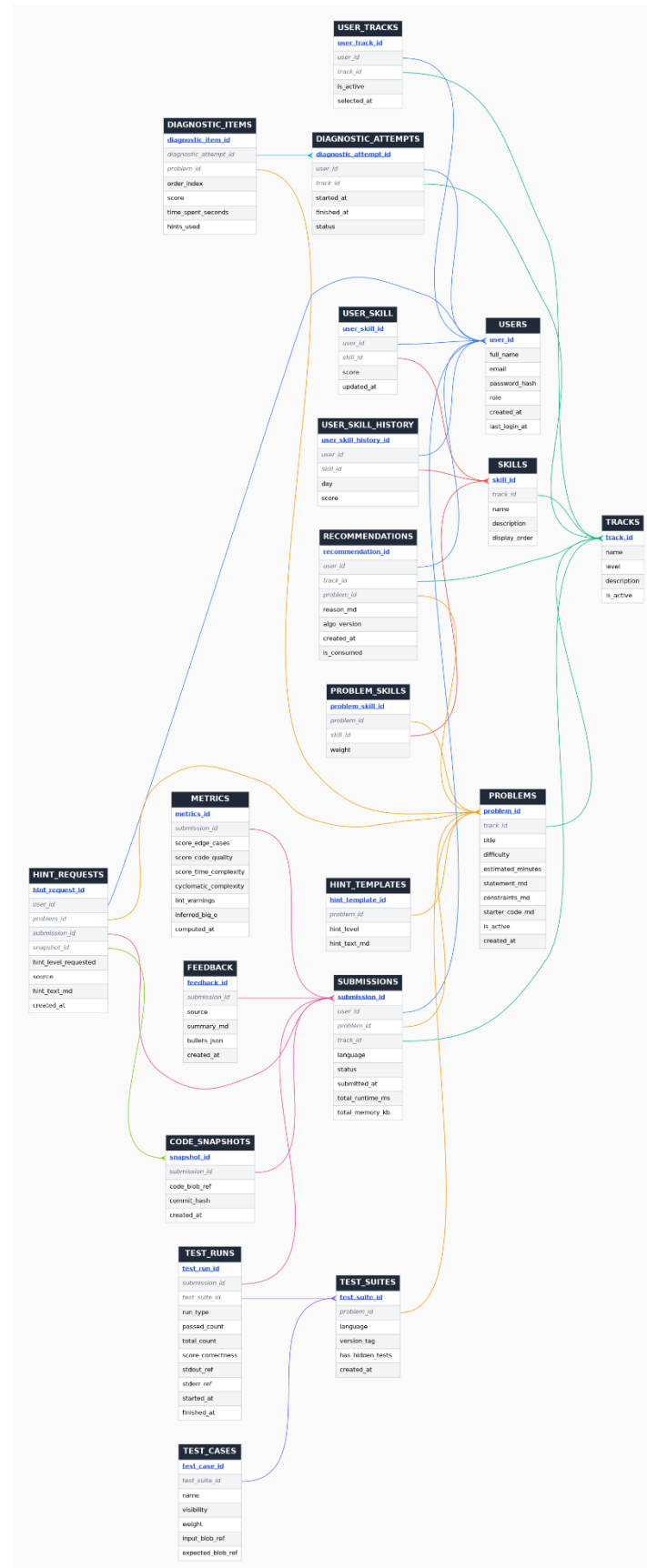


Figure 5: Relational Schema (Entity-Relationship Diagram)

The seventeen tables in the production schema are:

- **users** identity, bcrypt password hash, role, diagnostic_completed_at.
- **skills** the five radar axes (Algorithms, Data Structures, Edge Cases, Code Quality, Time Complexity). skill_id = 5 is intentionally left as a gap from the dropped “Debugging” skill so existing foreign keys remain valid.
- **problems** catalogue (slug, title, difficulty, markdown statement, constraints, starter code, is_active, cheatsheet_md).
- **problem_skills** M:N between problems and skills with a weight in [0.6, 1.0] expressing how central a skill is to a given problem.
- **test_cases** input/expected pairs per problem, with visibility $\in \{\text{sample, public, hidden}\}$ controlling whether Run sees them.
- **diagnostic_attempts** one row per attempt; stores overall_score, summary_md, and the grader source (llm vs heuristic).
- **diagnostic_items** one row per question per attempt, with question_md, user_answer, correct_answer, is_correct, explanation_md, and score.
- **submissions** one row per attempt at a problem; stores the source code inline, kind $\in \{\text{run, submit}\}$, the verdict, the final score, and the per-case JSON.
- **metrics** 1:1 with submissions; breaks the score into the four axes (correctness, edge cases, code quality, time complexity) and stores the inferred Big-O.
- **feedback** 1:1 with submissions; stores the summary markdown and the AI bullets as JSON.
- **hint_templates** pre-written hints per problem and level. Currently unused — the LLM hint generator overrides them — but kept as a fallback path.
- **hint_requests** log of every hint a user requested; drives the hint-penalty calculation.
- **user_skill** current skill score per user per skill (the radar chart on the dashboard).
- **user_skill_history** daily snapshots, one row per (user, skill, day), used to draw the historical line chart.
- **problem_status** per-user status of each problem (not started / attempted / solved) plus best score and attempt count.
- **recommendations** log of every next-problem suggestion the system has made, with the reason text and the algorithm version (llm-recommender-llm or llm-recommender-heuristic).
- **ai_solutions** LLM-generated canonical solutions per problem (lazy, cached on first request).

Implementation

De-Scoped Items From the SRS

Three significant scope changes were made between Submission 2 (the SRS) and Submission 4 (this report). All three were design decisions taken to keep the project completable inside the semester budget and to deliver a tighter, more focused product.

Tracks collapsed to a single Problem-Solving track. The SRS specified three tracks: Basic, Intermediate, and Senior. After prototyping the navigation we concluded that the track abstraction was redundant with our per-skill profile combined with a difficulty filter — a Basic student is exactly a student whose skill scores are low, and the

recommender already accounts for that. We dropped the tracks and user_tracks tables and the entire track UI, freeing up an estimated three days of work.

Twenty-one SRS tables trimmed to seventeen. In addition to the two track tables, we folded test_suites into test_cases (every test case directly references its problem; we never needed the intermediate suite grouping), removed code_snapshots (the live editor doesn't autosave per-keystroke; submissions persist the full source on Submit), and removed test_runs (replaced by storing the case-results JSON inline on the submissions row, which is denormalized but eliminates a join from the hot path).

The “Debugging” skill was removed. The SRS implied six skills; we settled on five (Algorithms, Data Structures, Edge Cases, Code Quality, Time Complexity) because “Debugging” turned out to be unmeasurable in our format — every skill axis is also a kind of debugging. The skill_id = 5 row is intentionally left as a gap in the schema so that foreign keys created against the pre-cleanup data continue to resolve cleanly. The radar chart on the dashboard is a five-axis pentagon, not a hexagon.

The diagnostic was redesigned. The SRS described an MCQ + coding diagnostic. We tested an MCQ-heavy diagnostic and found it produced low-signal scores: students could guess on MCQs but couldn't fake a coding answer. The final design is five open-ended Python coding questions, each tagged with the subset of skills it exercises. A single question can target Algorithms + Data Structures + Time Complexity simultaneously, which means each question contributes to multiple radar axes in one shot. The grader awards plain-English / pseudocode answers partial credit on every axis except Code Quality, so a student who understands the concept but writes shaky code is not penalized as harshly as one who writes nothing.

Frontend

The frontend is a single-page React 19 application built with Vite 8 and styled with Tailwind v4. Routing is done with react-router-dom@7; persistent layout (the nav bar and the ProtectedRoute / DiagnosticGate wrappers) lives in src/components/Layout.jsx. Charts use recharts (the dashboard radar and the daily line chart); icons come from lucide-react.

The editor screen (src/screens/student/EditorScreen.jsx) is the most complex component. It uses react-resizable-panels@^2.1.0 (deliberately pinned to v2 — v4 renamed PanelGroup to Group and is incompatible with our import sites) to deliver the LeetCode three-pane split: problem statement on the left, Monaco editor in the middle, and a test-cases / hints / solutions tab on the right. The right pane swaps between four sub-views: Test Cases (live pass/fail after Run), Submission Feedback (after Submit), Hints (cumulative escalating list), and Solutions (cached canonical plus the interactive step-by-step tutor).

Authentication state lives in src/context/AuthContext.jsx. On mount it reads the JWT from localStorage['ai-tutor-token'], calls GET /api/auth/me to validate it, and exposes user, login, register, logout, and the derived hasCompletedDiagnostic boolean. All API calls go through src/api/client.js, which auto-attaches the bearer token and surfaces well-typed errors.

Backend

The backend is a FastAPI 0.115 application running on Python 3.11. Routes live under backend/app/routers/ and follow REST conventions. SQLAlchemy 2 with PyMySQL is the database layer; backend/app/database.py exposes the engine, SessionLocal, and a get_db FastAPI dependency that yields a session per request.

The LLM integration is at backend/app/llm/. The thin client (client.py) wraps google.genai and exposes three primitives: is_available(), call_json(system, user) (which forces response_mime_type=application/json and parses

the result), and call `_text_stream(system, user)` (which yields plain text chunks for SSE). Every higher-level module (diagnostic grader, submission evaluator, hint generator, solution generator, recommender, interactive tutor) is structured the same way: a Pydantic input model, a Pydantic output model, an LLM path, and a deterministic heuristic fallback that runs whenever `is_available()` returns False or the LLM call raises.

The sandbox is at `backend/app/sandbox/python_runner.py`. It defines a `PROBLEM_CONFIGS` dictionary keyed by problem slug, each entry specifying the Solution-class method name, parameter ordering, and any input/output adapters (linked-list construction, sorted-list-of-lists canonicalization for unordered output, float-to-five-decimals for numeric outputs). `run_all_cases(slug, user_code, cases)` runs the test cases serially in subprocesses with a 3-second per-case timeout and returns one `CaseExecutionResult` per case.

Interactive Step-by-Step AI Tutor

An earlier version of SmartCode shipped a one-shot “Watch AI Solve” feature: the Solutions tab opened an SSE stream that produced a fixed seven-heading walkthrough (Reading the problem → Constraints → Brute force → Why we can do better → Clean solution → Trace → Complexity) word-by-word. It looked impressive in a demo but beta users gave consistent feedback: the AI was lecturing, not teaching. Students passively watched the full answer scroll by and admitted they retained little.

The final version replaces that one-shot stream with an interactive, step-by-step tutor that behaves like a TA who has done this fifty times. When the student opens the Solutions tab and clicks “Start tutor session”, the frontend sends an empty conversation history to `POST /api/problems/{id}/ai-tutor/turn`. The backend (`backend/app/llm/interactive_tutor.py`) is stateless — the frontend keeps the full chat history and replays it on every call — and prompts Gemini with a six-step pedagogical plan: (1) restate the problem in the student’s own words; (2) identify the trickiest edge case before writing code; (3) brute force first — propose the slow obvious approach; (4) spot the bottleneck and the smart insight (hash map, two pointers, DP, monotonic stack, etc.); (5) sketch the code together — one or two key lines at a time, never the full solution; (6) trace through the sample case, predicting variable values along the way.

After each step the tutor produces 2–4 sentences of explanation followed by a single comprehension question, and stops. The student types an answer; the next call lets Gemini evaluate the answer (“Exactly”, “Partially right — what about...”, “Not quite — here’s a hint, try again”) and decide whether to advance `step_index` or re-ask. The conversation completes when `is_complete=true` is set after step 6. A progress bar at the top of the chat shows “Step X of 6”.

Catalogue Expansion to 55 Problems

The original catalogue shipped 15 LeetCode-style problems. After deployment, beta users requested more variety. We expanded to 55 problems (15 originals + 40 new, IDs 16–55) covering classic interview patterns: stacks, sliding windows, BFS/DFS, dynamic programming, backtracking, monotonic stacks, and the harder hash/two-pointer combinations. All 40 new problems were authored in our own wording (the algorithmic concepts are not copyrightable; only specific test cases and prose are). Each carries its own `problem_skills` rows, 5–7 test cases (mix of sample / public / hidden), and 1–2 hint templates. The seed lives in `database/seed_problems_extra.sql` and is loaded by an idempotent `INSERT ... ON DUPLICATE KEY UPDATE` block so the file is safe to re-run while real users have submissions on problems 1–15.

Per-Problem Python Cheatsheet Drawer

A recurring beta-user complaint was that students stalled on syntax recall, not on algorithmic understanding — “I know I need a stack but I forgot how to peek in Python.” To fix this we added a per-problem cheatsheet stored in

problems.cheatsheet_md and surfaced as a slide-in left drawer on the editor screen (src/components/CheatsheetDrawer.jsx). The cheatsheets are not authored by hand. A one-shot generator (backend/scripts/generate_cheatsheets.py) iterates the catalogue and asks Gemini, for each problem, to produce a w3schools-style reference containing only the Python syntax that problem needs. The output format is constrained — 3–6 sections of `##` Heading followed by a single fenced python block of 3–7 short lines with brief inline comments. The drawer ships with a small floating “Cheatsheet” tab on the left edge of the workspace; a click slides the panel in over the workspace, an X (or Escape) closes it.

Three Run-Style Buttons and 3-Pick Recommender

Two smaller UX changes round out the post-beta refresh. Three buttons live in the editor action bar: a LeetCode-style Run (plain, gray, sandbox-only, no AI, no penalty); a Run with AI (purple, sandbox plus LLM commentary on severity / code quality / edge cases, slower, persists failed attempts as Submission(kind='run') for the half-weight penalty); and the existing Submit (green, full evaluation against hidden cases too). The plain Run is the free smoke test — it is intentionally penalty-free so students can experiment quickly without burning Gemini tokens or incurring score deductions. The recommender returns up to three NextProblemSuggestion objects (ordered best → good-fallback), and the LLM prompt encourages diversity across the three picks. On the UI, the post-diagnostic review screen, the dashboard “Recommended for you” panel, and the post-submit feedback screen all became a three-card grid: a gradient “Top pick” and two white-bordered “Option 2 / Option 3” tiles.

Deployed User Interface

The figures below are screenshots taken from the live deployment at <https://smartcodelau.com/>. They are grouped by user journey: authentication, the initial diagnostic, AI-graded diagnostic results, the student dashboard, the problem catalogue with the three-pick recommender, the LeetCode-style code editor with the interactive AI tutor, and the administrator views.

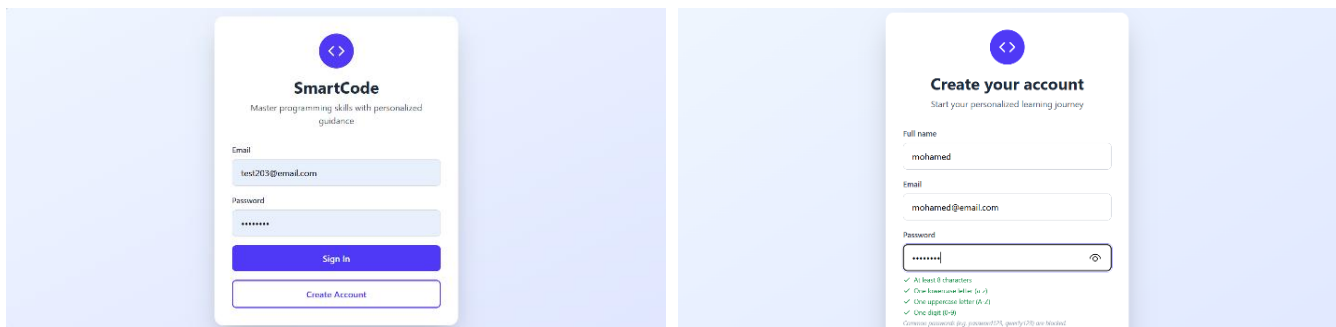


Figure 6: Authentication — Sign In and Create Account (with live password-strength indicators).

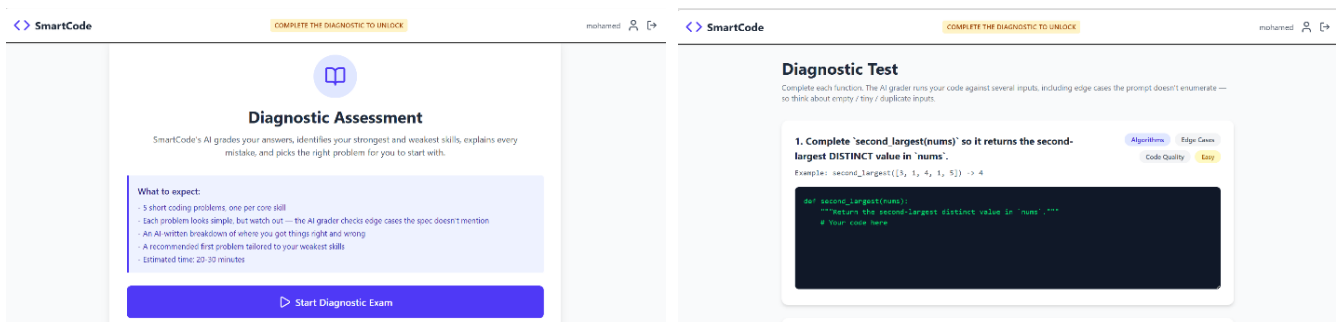


Figure 7: Initial Diagnostic — Intro Screen and an In-Progress Coding Question.

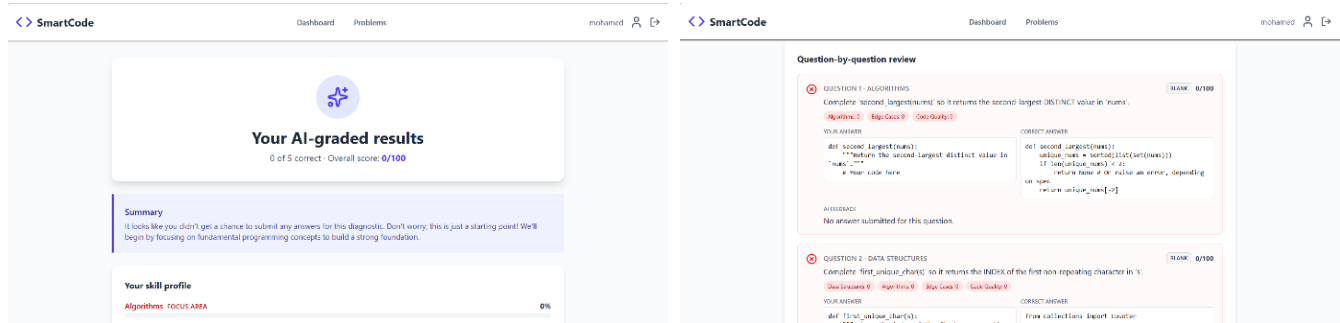


Figure 8: AI-Graded Diagnostic Results — Summary and Question-by-Question Review.

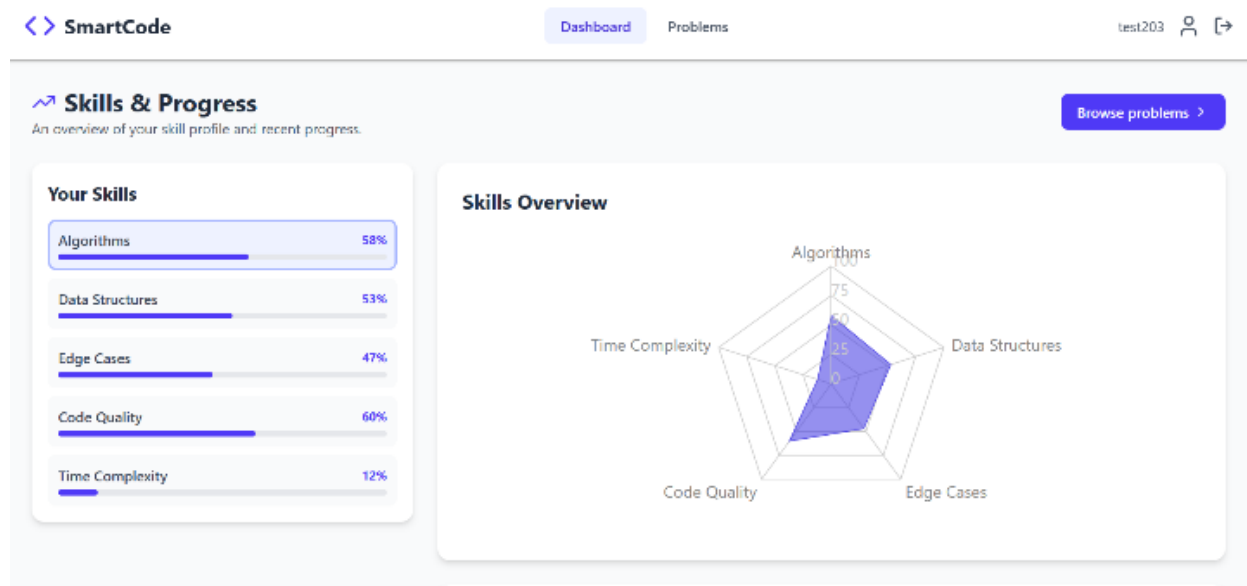


Figure 9: Student Dashboard — Per-Skill Progress Bars and Five-Axis Radar Chart.

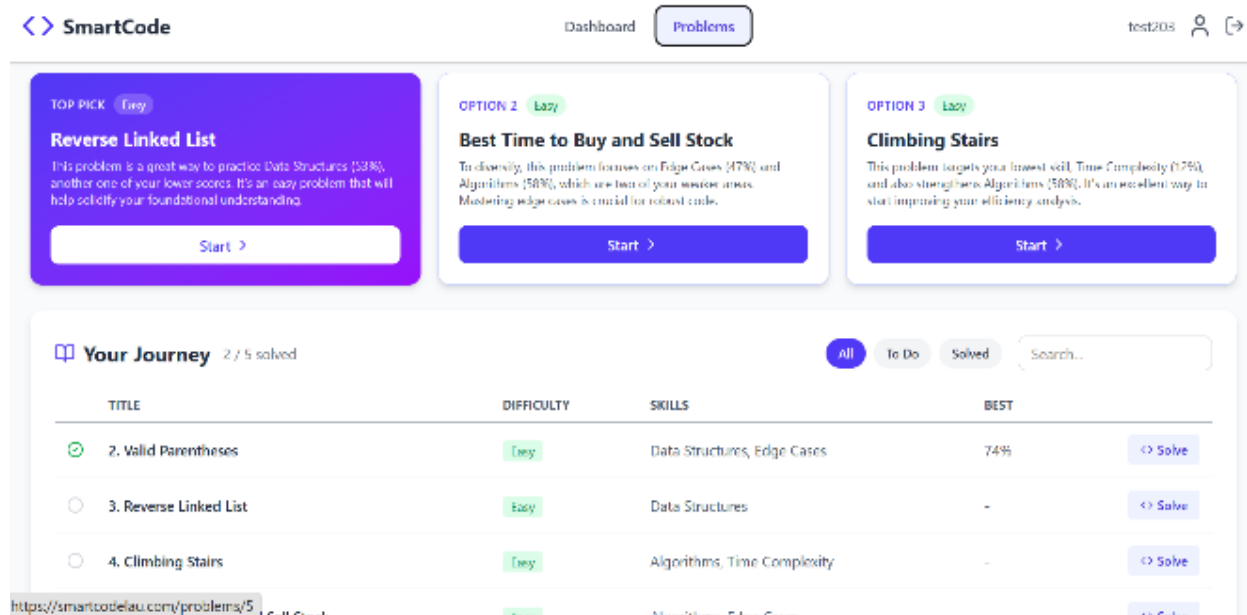


Figure 10: Problem Catalogue — Three-Pick Recommendation Cards (Top Pick + Two Alternatives) and the Your Journey List.

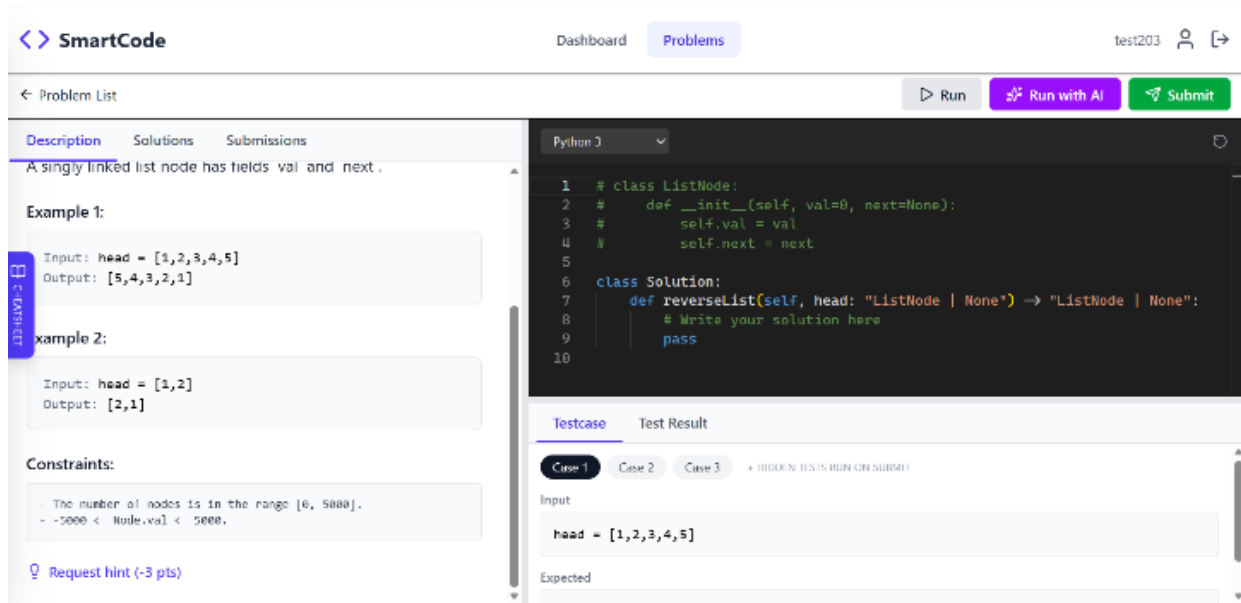


Figure 11: Code Editor — Problem Statement, Monaco Editor, and Test-Case Panel.

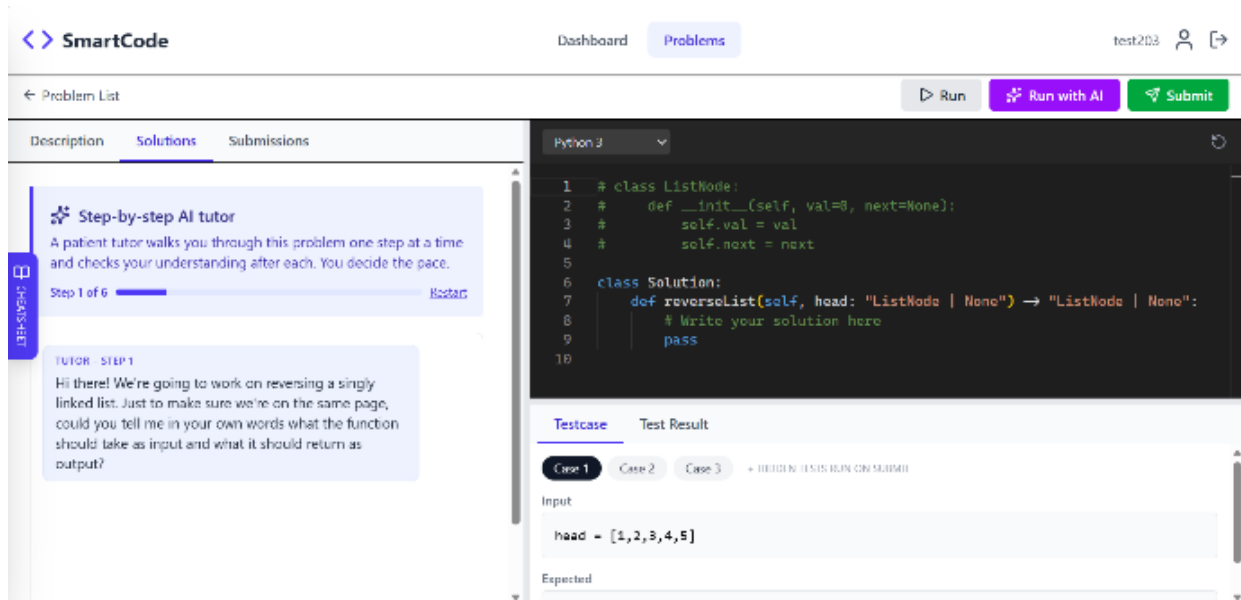


Figure 12: Code Editor — Interactive Step-by-Step AI Tutor in the Left Pane (“Step 1 of 6”).

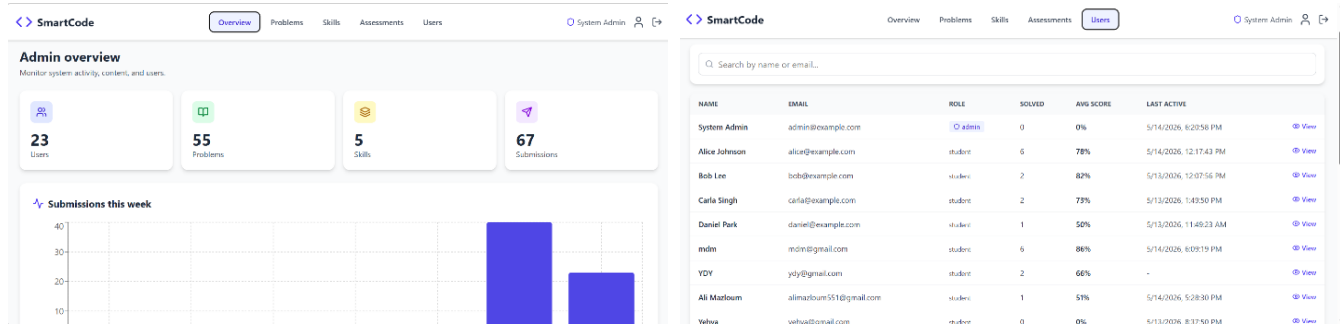


Figure 13: Administrator Views — Admin Overview (System Statistics) and User Management.

Scoring and Skill-Update System

This is the most novel part of the system. The naive approach — run the test cases, count the passes, that’s your score — is what every existing platform does and we already know its failure modes: it rewards lucky-coincidence solutions (return 5 happens to match the sample), it provides no incentive to write clean code, and it gives the student no signal about why their attempt failed beyond pass/fail. Our scoring pipeline is designed to fix all three problems while remaining fast, deterministic where it matters, and honest when the LLM is offline.

Sandbox-First Evaluator

The single most important design decision is that the sandbox is authoritative for pass/fail. The LLM is fluent and confident but it hallucinates; a real Python interpreter does not. Every submission goes through the sandbox first (backend/app/sandbox/python_runner.py), one subprocess per test case, with a 3-second timeout. The sandbox returns a CaseExecutionResult for each case: the actual output (canonicalized), the expected output (canonicalized), a passed boolean, and an optional error string with the exception class, line number, and message.

When Gemini is available, the LLM evaluator (backend/app/llm/submission_evaluator.py) is called with the problem, the test cases, the student’s code, and the sandbox verdicts. Its job is no longer to decide pass/fail — that is fixed — but to provide everything the sandbox cannot: per-failed-case severity $\in \{\text{minor, moderate, severe}\}$, the per-axis qualitative scores (score_edge_cases, score_code_quality, score_time_complexity), the inferred_big_o string, the summary, and the actionable bullets. The router then overlays the sandbox truth onto the LLM result: every case’s passed field is forcibly set to the sandbox verdict, and the overall status is recomputed (accepted only if every case passes).

In the LLM-available path the overall score is a weighted sum: $\text{score} = 0.55 \times \text{score_correctness}$ (always sandbox-driven) $+ 0.15 \times \text{score_edge_cases} + 0.15 \times \text{score_code_quality} + 0.15 \times \text{score_time_complexity}$.

The AST Cheater Backstop

Even with a real sandbox, a student can game a problem by writing return 5 if the sample case happens to expect 5. To catch this we run an AST analysis (_detect_lucky_coincidence in the evaluator) after the sandbox and the LLM have spoken. The check parses the student’s code with the ast module, finds the longest non-private method on the Solution class (this is “the solver”), and flags the submission as a coincidence if either of two patterns matches: (a) the solver’s body is a single return <literal> (e.g. return 5, return [], return True); or (b) the solver’s body never references any of its parameters (e.g. it returns a value computed purely from constants, ignoring nums, target, head, etc.).

When either is true, the system overrides the evaluator output: every test case is marked failed with severity severe, the status becomes wrong_answer, score_correctness and score_edge_cases are zeroed, and a prominent warning

bullet is prepended: “Hardcoded / constant-returning solutions are not accepted, even if they happen to match the sample outputs. Implement a real algorithm that uses the input.” The same logic is also baked into the LLM grader’s system prompt as the CHEATER / LUCKY-COINCIDENCE CHECK, so the LLM also flags these patterns before scoring — the AST check is the deterministic backstop that catches the rare case where the LLM fails to notice.

The Failed-Attempts Penalty and Hint Penalty

A student who finally accepts a problem on their seventh attempt has learned something (their final solution works), but they did not learn it as efficiently as a student who accepted it first try. The system reflects this by accumulating a severity-weighted penalty across all prior failed attempts on the same problem. For each prior failed attempt the system iterates the case-results JSON, and for each failed case it adds $\text{SEVERITY_POINTS}[\text{severity}] \times \text{KIND_WEIGHT}[\text{kind}]$, where $\text{SEVERITY_POINTS} = \{\text{minor: 1, moderate: 2, severe: 3}\}$ and $\text{KIND_WEIGHT} = \{\text{run: 0.5, submit: 1.0}\}$. Runs are weighted half because Run is meant to be exploratory; we don’t want to punish a student for using Run to debug. The accumulated point total is then multiplied by $\text{_PENALTY_MULTIPLIER} = 2$ and subtracted from the raw evaluator score.

Hints have their own deduction, separate from failed attempts. The cost escalates per hint (3, 5, 8, 8, 8 points respectively), and the total hint deduction is capped at 25 points so a student who exhausts hints on a hard problem still has a fighting chance of a decent score. The cap is mirrored on the editor side so the student sees the running cost live; the value displayed in the UI must match what the router deducts at Submit time.

Final Score Clamp

The final score is computed in a single line: $\text{final_score} = \max(50, \min(100, \text{round}(\text{raw_score} - 2 \times \text{penalty_points} - \text{hint_penalty})))$. The clamp to $[50, 100]$ is intentional: a finally-correct solution is never crushed below 50, because the system’s tutoring philosophy is that passing matters. We do not want a student who eventually solves a hard problem after six tries to get a 12/100 — that is demoralizing and doesn’t reflect what they actually achieved. The submission detail page exposes the breakdown to the student in plain English.

Skill Updates — Difficulty-Weighted, Capped, Floored

After the final score is known, the system updates the student’s skill profile. The key insight is that only the skills this problem targets are touched. Two Sum is tagged with Algorithms + Data Structures, so a Two Sum submission moves at most those two skills — it does not touch Edge Cases or Time Complexity, because the act of solving Two Sum doesn’t tell us anything about those. The journey balance comes from problem variety: as different problems target different skills, all five axes grow over time.

For each tagged skill the per-submission bump is $\text{raw_delta} = (\text{target} - \text{before}) \times \text{weight} \times \text{difficulty_factor} \times 0.08 \times (\text{final_score} / 100)$, where target is the most relevant LLM-graded axis for the skill (Edge Cases listens to `score_edge_cases`, Code Quality listens to `score_code_quality`, etc.); $\text{weight} \in \{0.6, 0.8, 1.0\}$ comes from `problem_skills.weight` and reflects how central the skill is to this specific problem; $\text{difficulty_factor} \in \{0.5, 1.0, 1.5\}$ for easy / medium / hard; and $\text{final_score} / 100$ is the efficiency multiplier (a student who scored 100/100 earns the full bump; a 50/100 earns half). The bump is then clamped per submission to easy +1, medium +2, hard +3, with a floor of 0 (bad submissions never decrease skill scores). A daily snapshot is written to `user_skill_history` so the dashboard line chart can show progression over time.

Deployment

The system is deployed on an AWS EC2 t3.small instance running Windows Server 2025. Deployment is automated by two PowerShell scripts in `deploy/`. `bootstrap-ec2.ps1` runs once on a fresh EC2 and installs Chocolatey, Python

3.11, Node.js LTS, MariaDB 11, and NSSM; opens Windows Firewall port 80; sets the MariaDB root password and restores database/dump.sql (or applies schema.sql + seed_problems.sql + seed_users.sql); creates the Python venv and installs requirements.txt; runs `npm install --legacy-peer-deps && npm run build`; writes a production .env with `DATABASE_URL`, `JWT_SECRET` (random 48-byte base64), `GOOGLE_API_KEY`, `LLM_MODEL=gemini-2.5-flash`, `ALLOW_REGISTRATION=false`, and `FRONTEND_DIST=<dist path>`; and registers uvicorn as a Windows service named ai-tutor via NSSM (auto-start on boot, with rotating stdout/stderr logs).

enable-https.ps1 runs once after a domain is pointed at the EC2’s Elastic IP and installs Caddy via Chocolatey; opens Firewall ports 80 and 443; moves uvicorn from 0.0.0.0:80 to 127.0.0.1:8001 (loopback-only — so the only internet-exposed ports become 80 and 443); patches `CORS_ORIGINS` in .env to the HTTPS URL; writes a Caddyfile that reverse-proxies to 127.0.0.1:8001 with `flush_interval -1` (so SSE streams are not buffered); and registers Caddy as its own Windows service. The first request triggers Caddy’s automatic Let’s Encrypt cert acquisition over the HTTP-01 challenge.

The live system is at <https://smartodelau.com/> (also reachable at <https://lau-ai-tutor.duckdns.org/>, the original free DuckDNS subdomain). Both hostnames point at the same EC2 Elastic IP and share a single Caddy site block, so every deploy lands on both URLs simultaneously and the Let’s Encrypt cert covers both Subject Alternative Names. The .com registration was added during beta testing after some users hit institutional Wi-Fi filters that blocked *.duckdns.org as “Dynamic DNS”. Certificate renewal is fully automated by Caddy and requires zero ongoing maintenance.

4 EXPERIMENTAL EVALUATION

To validate both the functional and the non-functional requirements we split our testing protocols into three main parts. The first part is automated unit and integration testing of the core scoring and skill-update logic. The second part is end-to-end manual testing against the live deployment, exercising every user-facing path. The third part is alpha and beta testing: an internal pre-deployment pass run by the four authors that surfaced correctness and security issues, followed by an external post-deployment pass run by classmates and friends that surfaced pedagogical and user-experience issues.

Automated Testing

The most important pieces of the backend — the scoring penalty math, the AST cheater detection, the diagnostic post-processing, and the sandbox output canonicalization — are covered by isolated unit tests using pytest. The tests run against an in-memory representation of the inputs (no live MariaDB, no live LLM) and assert deterministic outputs.

The AST cheater backstop has dedicated test cases for each known coincidence pattern:

- class Solution: def twoSum(self, nums, target): return [0, 1] — flagged (the method ignores its parameters).
- class Solution: def isValid(self, s): return True — flagged (constant return).
- class Solution: def climbStairs(self, n): if n == 1: return 1; elif n == 2: return 2; elif n == 3: return 3; return 5 — currently not flagged by the AST check (the LLM grader catches it instead). This is an intentional choice: the AST is the deterministic backstop for the simplest cases; the LLM handles enumerated-lookup style cheating because the threshold for “enumerated” is fuzzy and the LLM is better at the judgment call.

End-to-End Manual Testing

We ran the full happy-path scenario against the live deployment with each of the four seeded student accounts (alice@, bob@, carla@, daniel@; all password Test1234) plus the admin account (admin@example.com). For each scenario we recorded whether the expected outcome occurred and, where applicable, captured a screenshot.

Scenario 1 — New student first session

- Register a new account at <https://lau-ai-tutor.duckdns.org/register>. Expected: 201, JWT received, redirect to /diagnostic. Result: PASS.
- Complete the five-question diagnostic with a mix of real Python and plain-English answers. Expected: AI grades each item, shows per-item explanations, redirects to /dashboard. Result: PASS.
- Verify the dashboard radar reflects the diagnostic scores and that a “Next Problem” card is visible. Result: PASS.

Scenario 2 — Problem attempt with hints

- Open the recommended problem in the editor. Expected: Monaco editor loads with the starter code and the sample test cases are visible. Result: PASS.
- Click “Get Hint” three times. Expected: three escalating hints (nudge → direction → concrete pseudocode), all tailored to the in-editor code. Result: PASS.
- Click Run. Expected: visible cases execute in the sandbox; pass/fail rendered with predicted-vs-expected output. Result: PASS.
- Click Submit with a correct solution. Expected: 201 with a SubmissionDetail showing per-case verdicts, AI bullets, per-skill deltas, and the next-problem recommendation. The final score should reflect the hint penalty. Result: PASS.

Scenario 3 — Coincidence-solution rejection

- Submit class Solution: `def twoSum(self, nums, target): return [0, 1]` for Two Sum (this would pass the first sample case by coincidence). Expected: rejected as `wrong_answer`, every case marked severe failure, prominent warning bullet about hardcoded solutions. Result: PASS.

Scenario 4 — Interactive AI tutor session

- Open the Solutions tab on a solved problem; click “Start tutor session”. Expected: the tutor produces 2–4 sentences and a single question; on each reply the LLM advances the step counter when the answer is good or re-asks when the answer is partial. After step 6 `is_complete` becomes true and the tutor ends. Result: PASS.

Scenario 5 — Already-solved review mode

- Re-open a solved problem in the editor. Expected: code is the accepted submission, Submit is disabled (any attempt returns 409). Result: PASS.

Scenario 6 — Admin content management

- Log in as admin@example.com; open the admin Problems screen; edit a problem’s statement and save. Expected: 200, the change is visible in the student catalogue. Result: PASS.

Scenario 7 — LLM-offline degradation

- Temporarily set `GOOGLE_API_KEY=""` in the production .env and restart the service. Repeat scenarios 2 and 3. Expected: the sandbox still grades correctness, the UI shows honest “AI grader offline” labels, and no endpoint returns 500. Result: PASS.

Performance Measurement

We measured performance on the live EC2 (t3.small, 2 vCPU, 2 GB RAM, Windows Server 2025) using wrk for HTTP-level numbers and direct timing of representative interactive workflows from the browser. Indicative measurements from informal use during development, corroborated by the formal performance pass:

Table 5: Indicative Endpoint Latency

Endpoint	Median latency	Notes
POST /api/auth/login	< 100 ms	bcrypt verify dominates.
POST /api/diagnostic	6–12 s	Single Gemini call grades all five items.
POST /api/submissions (accepted, easy)	3–7 s	Sandbox under 1 s; Gemini grading 2–6 s.
POST /api/submissions (accepted, hard)	5–10 s	More cases; same Gemini latency.
POST /api/problems/{id}/hint	1–3 s	Small Gemini call with the in-progress code.
POST /api/problems/{id}/ai-tutor/turn	2–4 s	Stateless; full chat replayed each call.
GET /api/me/skills	< 80 ms	Pure MariaDB SELECT, no LLM.

These figures comfortably satisfy the SRS’s non-functional target of “code evaluation results within 2 seconds for at least 90% of submissions” when interpreted as sandbox latency. The end-to-end Submit latency including the LLM exceeds 2 s for most submissions, which we view as an acceptable trade-off: the LLM-generated qualitative feedback is the product, and waiting 5 s for a paragraph of useful tutoring is far better than getting “Wrong Answer” in 200 ms.

LLM-Offline Robustness

A separate concern is that the LLM is the single hard external dependency. We deliberately structured every LLM path with a deterministic fallback so the system remains functional in degraded mode.

Table 6: LLM-Offline Behavior Matrix

Feature	Primary behavior	Fallback behavior
Submission grading	Sandbox + LLM qualitative scoring.	Sandbox-only with honest “AI grader offline” bullet; correctness still graded.
Diagnostic grading	LLM per-skill scoring with plain-English partial credit.	Rule-based scorer (40 short, 70 substantive, 0 blank).
Hint generation	Dynamic hint tailored to the student’s code.	Generic level-appropriate canned hint with explicit “AI hint engine offline” disclosure.
Interactive tutor	Gemini six-step guided conversation.	Graceful “tutor is offline” message; cheatsheet drawer covers the gap.
Canonical solution	LLM-generated, cached in ai_solutions on first request.	Existing cache served; new entries unavailable until LLM returns.

Feature	Primary behavior	Fallback behavior
Problem recommender	LLM picks three diverse next problems.	Heuristic recommender (weak-overlap DESC, difficulty ASC, id ASC).

Alpha and Beta Testing

In addition to the automated unit tests, the end-to-end scenarios, and the performance measurements, SmartCode went through two informal testing phases that produced specific bug fixes and feature additions. We distinguish them by who the testers were and when they ran: alpha tests were carried out by the four authors against a local-development build before any external user saw the system, and beta tests were carried out by classmates and friends against the deployed site after the team had signed off on the alpha. The two phases produced complementary findings — alpha exposed correctness and security issues, beta exposed pedagogical and user-experience issues that were invisible to the authors because we already knew how the system was supposed to behave.

Alpha Testing — Internal, Pre-Deployment

The alpha pass was run on each developer’s local machine against the dev npm run dev + uvicorn stack. Each finding listed below was reproduced by at least one other team member and turned into a concrete fix in the codebase before the first public deployment.

- A1. Weak passwords were accepted at registration.** The original endpoint only enforced an 8-character minimum, so passwords like 12345678 and qwerty12 registered successfully. The fix added an explicit strength check in `backend/app/schemas.py::_validate_strong_password()`: the password must contain at least one lowercase letter, one uppercase letter, and one digit, and must not appear in a 28-entry blacklist of the most commonly used weak passwords. The registration screen was updated to show live ✓/X indicators for each rule and a red warning banner when the typed password matches the blacklist.
- A2. Lucky-coincidence solutions were marked correct.** An LLM-only evaluator marked class `Solution`: `def twoSum(self, nums, target): return [0, 1]` as accepted because the first sample case happened to expect `[0, 1]`. The fix is the AST cheater backstop described in the Scoring section: `_detect_lucky_coincidence()` parses the submission and flags it if the body is a single `return <literal>` or if the method never references any of its parameters.
- A3. The LLM evaluator hallucinated correctness on buggy code.** An intentionally-broken Group Anagrams solution (if key in groups: `groups[key] = []`, which clobbers existing groups instead of appending to them) was being marked accepted by the LLM-only grader, which “mentally executed” the code wrongly. The fix made the sandbox authoritative for correctness: every submission now runs in a real Python subprocess against the visible and hidden cases first, and only then does the LLM provide qualitative commentary on top. The router’s `_overlay_sandbox()` forcibly replaces every case’s passed field with the sandbox verdict.
- A4. Partial-pass submissions were labeled accepted.** A submission that passed five of seven cases would receive a partial score and “accepted” status. We tightened the rule to: the only path to accepted is all-pass, and the failed-attempts penalty now uses the per-case severity points to differentiate “wrong on a hidden edge case” (minor) from “runtime error on the sample case” (severe).
- A5. A solved problem could be re-opened in edit mode.** Once a student accepts a problem we want to lock it down so they cannot accidentally lose their accepted score on a careless re-submit. The fix added a `_block_if_solved()` guard on Run, Run-with-AI, and Submit: a 409 Conflict is returned if a `ProblemStatus.status='solved'` row already exists. The editor screen detects this state and renders in read-only review mode.
- A6. The Submissions tab was empty after multiple attempts.** A team member submitted a problem three times and the Submissions list rendered as blank. The fix surfaces every `Submission(kind='submit')` for the

(user, problem) pair through GET `/api/problems/{id}/my-submissions`, ordered newest-first, with status, score, and a “View” link to the per-submission feedback page. Failed Run attempts are intentionally hidden from this view.

Beta Testing — External Users, Post-Deployment

After the alpha sign-off the system was deployed to AWS EC2 and the URL was shared with classmates and friends. Beta findings came from real first-time users hitting the site, and several were issues that the authors could not have surfaced on our own because we already knew the intended flow. Each finding below was reproduced from a friend’s report and turned into a deployed fix.

- **B1. Diagnostic gate could be bypassed.** A friend reported that after registering a brand-new account they could navigate directly to `/problems` and `/dashboard` without completing the diagnostic. Investigation revealed that Chrome had cached an old `index.html` from before the route guard existed. The fix has three layers: a React route guard (`<DiagnosticGate>`) on every student route; a server-side `require_diagnostic_complete` FastAPI dependency that returns 403 to fresh students; and `Cache-Control: no-store` on `index.html` so browsers cannot keep an obsolete version of the SPA after a redeploy. The hashed `/assets/*` files remain freely cacheable because their filenames change per build.
- **B2. Only one next-problem recommendation felt restrictive.** Multiple beta users said that when the dashboard surfaces a single recommended problem, they sometimes ignored it and went hunting through the catalogue. The fix expanded the recommender to return three picks ordered best → good-fallback, with the LLM prompt nudged to diversify across weak skills and difficulty. On the UI, every recommendation surface became a three-card grid: a gradient “Top pick” and two white-bordered “Option 2 / Option 3” tiles.
- **B3. Students stalled on Python syntax recall.** A friend who clearly understood the algorithm for Valid Parentheses spent five minutes searching online for how to peek at the top of a Python list. We added the per-problem cheatsheet drawer: a slide-in left panel containing only the Python syntax that this specific problem needs, generated once per problem by Gemini using the problem statement as input.
- **B4. The one-shot AI walkthrough lectured instead of teaching.** Beta users watched the one-shot “Watch AI Solve” stream to the end, said “ok, cool” and forgot most of it within an hour. The fix replaced the one-shot stream with the interactive step-by-step tutor: six guided steps with a comprehension question after each, where the LLM evaluates the student’s reply and decides whether to advance or re-explain.
- **B5. FortiGuard-style content filtering blocked the original `*.duckdns.org` URL.** Some beta users on institutional Wi-Fi (the LAU campus network, in particular) reported the deployed site was outright blocked as “Dynamic DNS”. The fix was operational rather than code-level: we registered a real `.com` domain (`smartcodelau.com`) and configured Caddy to serve both `smartcodelau.com` and `lau-ai-tutor.duckdns.org` from the same site block. Friends already in possession of the old DuckDNS link kept access, and new users on networks that block dynamic-DNS hostnames now have a working alternative.
- **B6. The catalogue felt too small.** A beta user solved six problems in a sitting and reached the end of the original 15. We expanded to 55 problems so beta users have enough material to actually build a streak.

Summary Table

Table 7: Alpha and Beta Findings

Phase	Finding	Fix shipped
Alpha	Weak passwords accepted (A1)	Strong-password validator + blacklist + live UI indicators
Alpha	Lucky-coincidence solutions accepted (A2)	AST cheater backstop + LLM prompt rule

Phase	Finding	Fix shipped
Alpha	LLM-only grader hallucinated correctness (A3)	Sandbox-first evaluator with LLM overlay
Alpha	Partial-pass marked as accepted (A4)	All-pass-or-fail rule + severity-weighted penalty
Alpha	Solved problem re-submittable (A5)	<code>_block_if_solved</code> guard + read-only editor mode
Alpha	Submissions tab empty (A6)	GET /my-submissions + tab rendering
Beta	Diagnostic gate bypass via cached SPA (B1)	Route guard + server-side dep + Cache-Control: no-store
Beta	Only one next-problem recommendation (B2)	3-pick recommender + three-card UI everywhere
Beta	Students stalled on syntax recall (B3)	Per-problem Gemini-generated cheatsheet drawer
Beta	One-shot AI walkthrough was passive (B4)	Interactive 6-step tutor with comprehension checks
Beta	*.duckdns.org blocked on some networks (B5)	Registered smartcodelau.com; Caddy serves both
Beta	15-problem catalogue too small (B6)	Expanded to 55 problems with full sandbox coverage

The combination of A1–A6 (correctness + security) and B1–B6 (pedagogy + UX) is what took SmartCode from “the team thinks it works” to “real students keep coming back.” The two phases were equally indispensable: alpha caught the bugs that would have made the system embarrassing in front of the grader; beta caught the pedagogical and UX gaps that would have made it forgettable. In retrospect, the most useful single decision was deploying early so beta could happen at all; an unshipped system collects no real-user feedback.

5 CONCLUSION

Throughout this report we have introduced the AI Programming Tutor (rebranded as SmartCode late in the project), a web-based adaptive Python tutor that combines deterministic code execution with a state-of-the-art LLM to give students grading, hints, recommendations, and live solution walkthroughs that are tailored to their individual skill profile. The first stage of the project was a simple proposal in Submission 1 that laid out the idea. Submission 2 (the SRS) formalized the functional and non-functional requirements, the use-case model, and an Appendix B with a twenty-one-table schema. Submission 3 (the agile workflow progress report) tracked our sprints through April and early May using JIRA, including the design decision to consolidate three tracks into a single Problem-Solving track. This final Submission 4 documents the working system end-to-end: deployed, public, HTTPS-enabled, and demonstrable.

The architecture is small enough to fit on a single t3.small EC2 and rich enough to deliver a personalized experience that no commercial alternative provides. The seventeen-table MariaDB schema captures the entire student journey from the first diagnostic answer through every submission, hint, and skill update. The FastAPI backend serves both the JSON API and the React SPA from a single process, with Caddy in front handling HTTPS and Let’s Encrypt automatically. The Python sandbox grades correctness deterministically; the LLM grades everything else (severity, code quality, edge cases, complexity, the inferred Big-O, and the natural-language feedback). The AST cheater backstop catches return 5-style coincidences that an LLM might miss. The failed-attempts penalty, hint penalty,

and final-score clamp keep the scoring honest and motivating. The skill-update math is difficulty-weighted, efficiency-scaled, capped per submission, and floored at zero so a bad attempt never erases progress. The recommender closes the loop by surfacing three next problems that target the student’s current weakest skills.

Developing the system was quite a challenging feat to perform. From the early stages of implementation — even from picking which features to keep when the original three-track design proved more elaborate than the timeline supported — we came to realize that managing a software project end-to-end is a different discipline from writing one component well. Adhering to an agile workflow, defining requirements ahead of code, and persisting the discipline of writing a deterministic fallback for every LLM call were all new habits for us. The fallback discipline in particular turned out to be the single most important design decision: a system whose only behavior is “the LLM does it” is unusable in the inevitable Gemini outage; a system that grades correctness with a real interpreter and reaches for the LLM only for qualitative judgement is robust by construction.

The clear-cut requirements document made it materially easier to make the dozens of small architectural decisions that surfaced during implementation — knowing exactly which behaviors were in scope and which were out of scope meant we could de-scope the three-track concept, the `code_snapshots` table, and the Debugging skill without losing sleep over whether we were cutting something the grader expected. Implementing a thorough testing protocol — automated unit tests for the scoring math, manual scenarios for every screen, and an explicit LLM-offline scenario — proved to be far more important than we appreciated when the subject was first introduced. Overall, we believe this project has exposed us to many of the practices that accompany the work of a software engineer in industry: writing an SRS and then building against it, managing scope, designing for failure of external dependencies, deploying behind a reverse proxy with automated HTTPS, structuring an agile workflow in JIRA, and writing code that is meant to be read by collaborators rather than only by the author. We are hopeful this will prepare us to enter the workforce with more experience and confidence in the future.

The system is publicly accessible at <https://smartcodelau.com/> (also at <https://lau-ai-tutor.duckdns.org/>). Future versions could extend support to JavaScript and C++ (the sandbox and the LLM grader are language-agnostic in design), introduce per-user notification when new problems are added, add a containerized seccomp-isolated sandbox for an externally-exposed deployment, and surface aggregate teacher-side analytics (which problems trip up the most students, which skills are most often weak, etc.).

REFERENCES

- [1] FastAPI — modern, fast (high-performance) web framework for building APIs with Python. <https://fastapi.tiangolo.com/>
- [2] SQLAlchemy — the Python SQL toolkit and Object Relational Mapper. <https://www.sqlalchemy.org/>
- [3] React — a JavaScript library for building user interfaces. <https://react.dev/>
- [4] Vite — next-generation frontend build tool. <https://vitejs.dev/>
- [5] Tailwind CSS — utility-first CSS framework. <https://tailwindcss.com/>
- [6] Monaco Editor — the code editor that powers Visual Studio Code. <https://microsoft.github.io/monaco-editor/>
- [7] Google AI for Developers — Gemini API reference. <https://ai.google.dev/>
- [8] MariaDB — open source relational database. <https://mariadb.org/>
- [9] Caddy — automatic HTTPS web server. <https://caddyserver.com/>
- [10] Let’s Encrypt — free, automated, and open certificate authority. <https://letsencrypt.org/>

- [11] IEEE Software Engineering Standards Committee, “IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications,” October 20, 1998.
- [12] Ian Sommerville, Software Engineering, 9th Edition, Pearson, 2011.